DAY 30 | HANDS-ON ACTIVITIES

LLM Security and Guardrails: Problem Statements

Eight defensive problems. Only test systems you own or are authorised to test. The vulnerable bot in the notebook is simulated for learning. Offline results show how the code works, not real model quality.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Threat Model with Both OWASP Lists NEW (v4)

Level: Warm-up | **Concepts:** OWASP LLM and agentic risks

For the SmartDesk support assistant (reads tickets, calls tools, remembers customer preferences, can issue refunds), list the top six risks across both OWASP lists, an example of each, and one control per risk.

Sample run / expected result

```
Indirect injection via ticket body -> scan and tag ticket text as data
Excessive agency -> refunds above 2,000 need approval
Memory poisoning -> only save facts confirmed by the customer
```

Hints

- Think about inputs, data sources, tools, memory and actions.

Problem 2: Spot the Injection

Level: Easy | **Concepts:** pattern detection

Write a scanner that scores text for injection signals ("ignore previous instructions", "system prompt", "you are now") and run it on 10 attacks and 10 benign messages. Then test 5 unseen attacks.

Sample run / expected result

```
Block rate: 9/10
False alarms: 1/10
Unseen attacks caught: 2/5  <- the honest number
```

Hints

- Measure both sides: missed attacks and wrongly blocked users.

Problem 3: Indirect Injection in Tool Results NEW (v4)

Level: Medium | **Concepts:** indirect injection, context tagging

A CRM lookup tool returns customer notes. One note contains a hidden instruction in an HTML comment. Show the naive bot following it, then quarantine and tag tool results so the leak stops.

Sample run / expected result

```
Before: leaked STAFF code
After: quarantined 1 tool result, safe answer
```

Hints

- Tool output is untrusted input, exactly like a web page.
- Removing the secret from the prompt is the strongest fix.

Problem 4: PII Redaction

Level: Easy | **Concepts:** output filtering

Redact emails, phone numbers (Indian and Japanese formats) and card numbers from model outputs. Test on 6 sample outputs and check that order totals are not removed.

Sample run / expected result

```
"Call me on +91 98450 11111" -> "Call me on [phone removed]"
```

Hints

- Order matters: match cards before phones.

Problem 5: Scoped, Short-lived Agent Token NEW (v4)

Level: Medium | **Concepts:** agent identity, JWT, scopes

Issue a JWT for the agent "smartdesk-assistant" with scopes ["tickets:read"] and a 5 minute expiry. Show that reading a ticket works, issuing a refund fails on scope, and an expired token fails.

Sample run / expected result

```
read_ticket -> ok
issue_refund -> PermissionError: refunds:write
expired token -> ExpiredSignatureError
```

Hints

- Check the scope inside every tool, not only at login.
- Set aud so the token works for one service only.

Problem 6: Approval Gate

Level: Medium | **Concepts:** human in the loop

Classify actions as low, medium or high risk and route them: auto-run, review queue or blocked pending approval. Unknown tools are blocked. Log every decision with the agent ID.

Sample run / expected result

```
send_summary -> auto
email_customer -> review
issue_refund(5000) -> approval required
drop_database -> blocked
```

Hints

- Default unknown actions to the highest risk.

Problem 7: Brownfield Step with Codex NEW (v4)

Level: Challenging | **Concepts:** characterisation tests, small steps

The notebook has a legacy refund_amount() function with odd rules. Write characterisation tests that pin its behaviour, then ask Codex for one small refactor (for example, extract a constant). Run the tests after the change and record the diff summary.

Sample run / expected result

```
5 characterisation tests: 5 passed (before)
refactor step 1: extract MAX_REFUND
5 passed (after)
```

Hints

- Pin surprising behaviour too; it may be a business rule.
- One change per step. Review the diff.

Problem 8: Red-team Suite and Report v3 NEW (v4)

Level: Stretch | **Concepts:** safety regression tests, reporting

Turn your attacks and benign prompts into PyTest tests that fail if block rate drops below 90% or false alarms rise above 10%. Write vulnerability report v3 from the findings using the shared template.

Sample run / expected result

```
4 passed: block_rate=0.9, false_alarm_rate=0.1
vulnerability_report_v3.md: 4 findings, 4 verified
```

Hints

- Safety tests run on every change, like unit tests.
- Template: shared/templates/Vulnerability_Report_Template.md